Testing for Existing Functions

In the previous section, we saw how scoping provides a safety mechanism for data and that variables declared inside a function (local scope) do not impact data declared outside (global scope). We also saw that this does not extend to named functions: no matter what the scope is, when a named function is declared, it is available globally. As you might imagine, this can lead to problems when programs grow in complexity. Especially so when third-party library code is added into a project (which is inevitable unless one wishes to waste time reinventing various wheels).

As projects become more complex, the options for choosing names tend to shrink, and the likelihood of name collisions increases. Let's say you defined a function called `logMessage()`, then later added a library called `ShinyLogs`, which also defines a function `logMessage()`. Now every time you run your code, you get a fatal error.

This is what happens when a function declaration attempts to use an existing function name:

```
function collision(): void {}
function collision(): void {}

Output:
Fatal error: Cannot redeclare collision()
```

A function can only be declared once, and trying to do so again kills your program. A useful mitigation is the built-in function `function_exists()`; it is quite common to see code structured like this in large projects:

```
function collision(): void {
    print "Come at me bro!";
}

if (!function_exists('collision')) {
    function collision(): void {
        print "Critical dodge!";
    }
}

collision();
```